

TrieHard: Auto Word Complete System Using Trie Data Structure

Muhammad Hadi
Syed Ebad Ali
Eishal Keshwani

1 Problem Statement and Significance

In today's digital world, users expect fast typing assistance while writing messages, searching online, or coding. Typing full words manually can take time and effort. Because of this, auto-complete systems are very useful.

The goal of this project is to build an **Auto Word Complete System** using the Trie data structure. This system suggests words based on the letters typed by the user. As the user types a prefix, the system quickly shows possible matching words.

This project is important because auto-complete systems are widely used in:

- Search engines
- Mobile keyboards
- Text editors
- Spell checking tools

By using an efficient data structure like Trie, the system becomes faster and more useful for real-world applications.

2 Data Structure and Algorithm Justification

In this project, we used the **Trie data structure**, also called a Prefix Tree.

A Trie stores words in a tree-like structure where:

- Each node represents one character
- Words sharing the same prefix share the same path

For example, words like:

- cat

- car
- call

share the same starting characters in the Trie.

We selected Trie because:

- It supports fast searching of words
- It quickly finds words with a common prefix
- It is memory efficient when many words share prefixes
- It works very well for auto-complete systems

The main operations used in our project are:

- Insert word
- Delete word
- Search word
- Suggest words using prefix search

These operations help the system provide real-time suggestions to the user.

3 Time Complexity and Memory Analysis

Time complexity is important to measure how fast the system works.

In the Trie data structure:

- Inserting a word takes: $O(L)$
- Searching a word takes: $O(L)$
- Prefix searching takes: $O(L)$

Here, L represents the length of the word.

This means the performance depends only on the length of the word, not the number of words stored in the system. Because of this, Trie is faster than linear search when handling large dictionaries.

Memory usage in Trie is slightly higher than simple lists because each character is stored as a node. However, memory is saved when many words share the same prefix.

For example:

“car”, “card”, and “care” share common nodes.

This makes Trie both fast and efficient for auto-complete applications.

4 Conclusion

In this project, we successfully developed an Auto Word Complete system using the Trie data structure.

The system allows users to:

- Get word suggestions quickly
- Add new words
- Delete words
- View previously used words

The Trie data structure improved the speed and efficiency of searching words with common prefixes.

This project shows how data structures learned in class can be used to solve real-world problems.